CODEPATH*ORG

Recursion

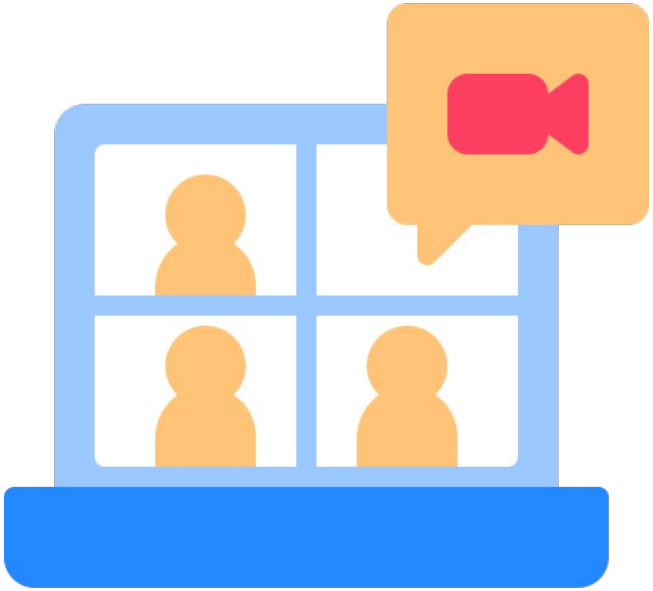
TIP101

Agenda

1	Welcome 👏🏽	10 minutes
2	Recursion	35 minutes
3	**BREAK**	5 minutes
4	Collaborative Problem Solving	60 minutes
5	Wrap Up	10 minutes

10 minutes

Welcome! 



Turn on your cameras

Fill out the entry ticket

Icebreaker

5 minutes

🌞**Question:**

If you could had to choose between only seeing the sunrise or only seeing the sunset, which would you choose and why?

💬**Answer:**

Share your answer in the Zoom chat below. What kinds of pets do you have if any? If you don’t have any, share what type you would like to have, if any.

Your Feedback

Glows

+ Mock interviews were a hit!
+ Students feeling confident after watching their peers

Grows

- Timing is tight on interview days

- Students looking for additional mock interview opportunities.

Announcements

- **Peer to Peer Mock Interviews**
  - All students will participate in 2 rounds.
    - 1 as the interviewer
    - 1 as the interviewee
  - Students must be camera ready and able to share their screen while solving the problem on VS Code.
  - Questions will be shared a little bit ahead of time.

Tool: VS Code Live Share

- Real-time collaboration - see your peers' edits live on your device!
- **VS Code Live Share** is available through a VS Code Extension -
  - How to install a VS Code extension
  - VS Code Live Share Extension
- **Note:** We expect screen sharing in rooms, even if you utilize VS Code Live Share. During technical interviews, you may be asked to screen share, pair program using a collaborative editor like Live Share, or both. Screen sharing also helps CodePath staff quickly catch up on your progress in breakout rooms without interrupting your workflow.

35 minutes

Recursion

Recursion is when a method calls itself.

```
def recursive_sum(num):
    if num <= 1:
        return num
    return num + recursive_sum(num - 1)

# Example usage
result = recursive_sum(5)
print(result)  # Output will be 15
```

The **base case** is the instance where a recursive method will **return** a value rather than calling itself.

The **recursive case** is the instance where the recursive method calls itself.

```
def recursive_sum(num):
    if num <= 1:
        return num
    return num + recursive_sum(num - 1)
```

```
def iterative_sum_while(num):
    total = 0
    while num > 0:
        total += num
        num -= 1
    return total
```

The **base case** comes from the part of the iterative method that **stops the repetition**. It occurs when a **certain condition is met**.

What happens when we forget the base case?

```
def recursive_sum(num):
    if num <= 1:
        return num
    return num + recursive_sum(num - 1)
```

```
def iterative_sum_while(num):
    total = 0
    while num > 0:
        total += num
        num -= 1
    return total
```

The **recursive case** is the part of the iterative method that **repeats**.

The **recursive call** has its own set of **local variables**, including the **formal parameters**.

```
def recursive_sum(num):
    if num <= 1:
        return num
    return num + recursive_sum(num - 1)

# Example usage
result = recursive_sum(5)
print(result)  # Output will be 15
```

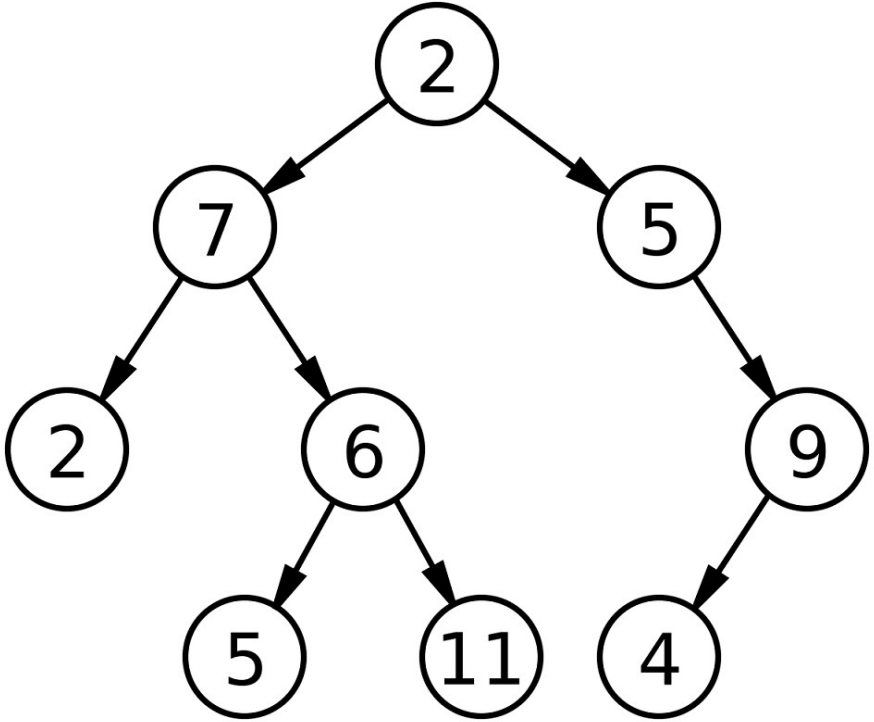
The **parameter values** capture the **progress of a recursive process**, just like how **loop control variables** capture the **progress of a loop**.

Why Recursion?

Sometimes, it's simpler than the iterative approach, where you need to divide and conquer that have a clear base case.

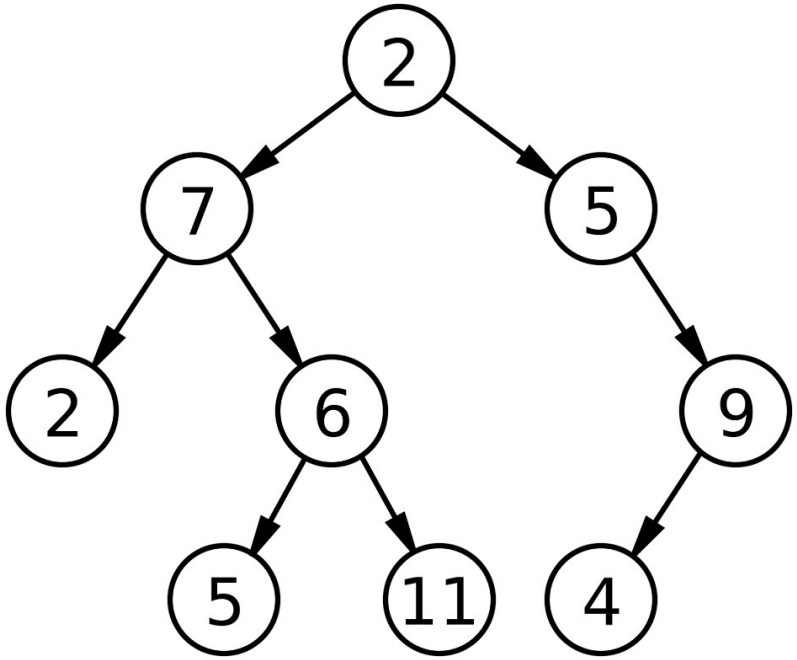
* Sorting or binary search (divide and conquer)
* Permutations and combinations
* Tree/graph traversal

Binary tree (coming next)



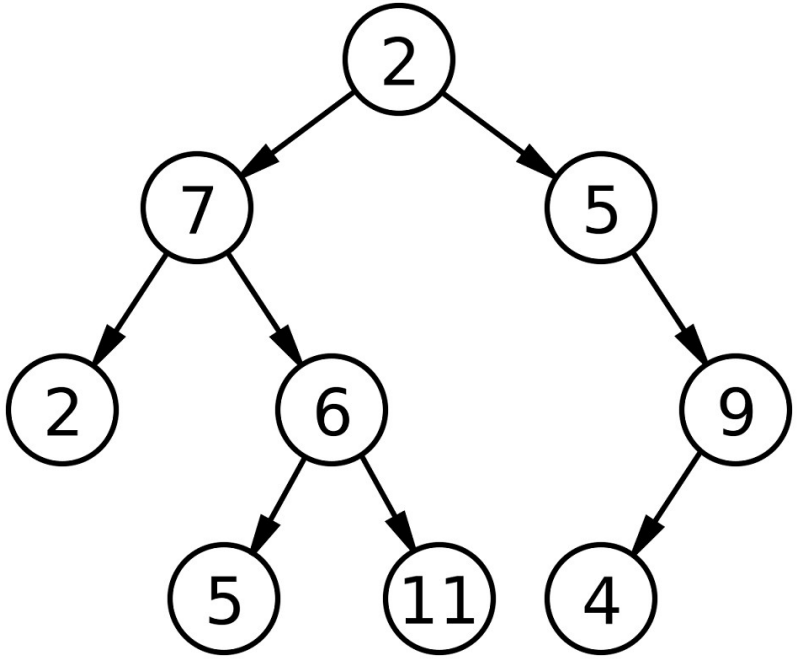
Print binary tree (recursive)

```
def print_tree(node):
    if node:
        print(node.value)
        print_tree(node.left)
        print_tree(node.right)
```



Print binary tree (iterative)

```
def print_tree(root):
    stack = []
    current = root
    while stack or current:
        while current:
            stack.append(current)
            current = current.left
        current = stack.pop()
        print(current.value)
        current = current.right
```



Why *Not* Recursion?

- It's confusing sometimes!
- Iterative solutions are usually easier to understand and debug for students.
- In reality, recursion is more commonly used to test conceptual understanding in interviews rather than in production code, where iterative solutions are usually preferred for clarity and traceability.

Example: Factorial

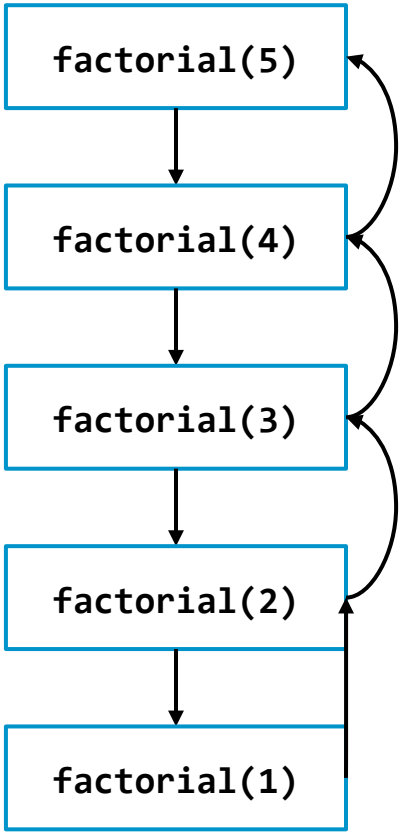
Implement a method to calculate the factorial. What is the base case?

- 5! = 5 * 4 * 3 * 2 * 1
- 4! = 4 * 3 * 2 * 1
- 3! = 3 * 2 * 1
- 2! = 2 * 1
- 1! = 1

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)
```

What is the result of `factorial(5)`?

120




```
return 5 * factorial(4)
```

5 * 24 = 120

```
return 4 * factorial(3)
```

4 * 6 = 24

```
return 3 * factorial(2)
```

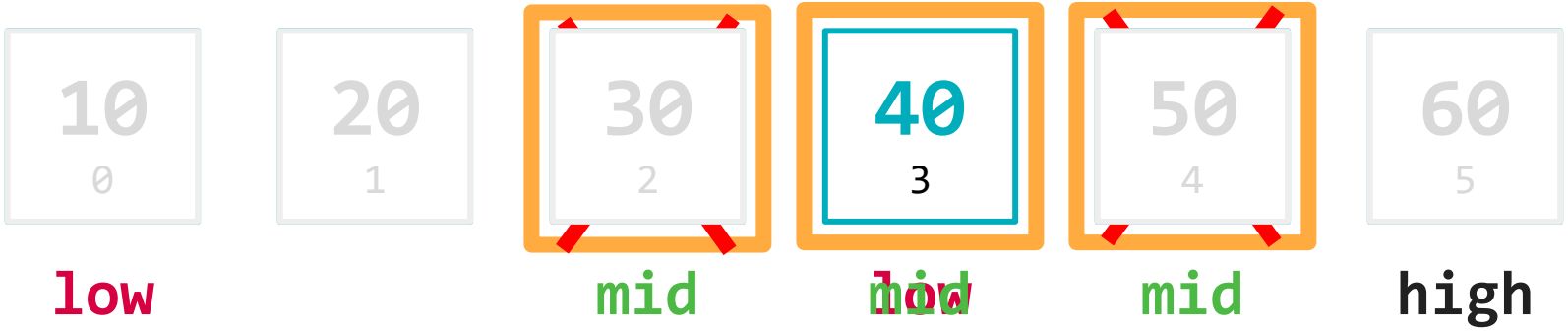
3 * 2 = 6

```
return 2 * factorial(1)
```

2 * 1 = 2

```
return 1
```

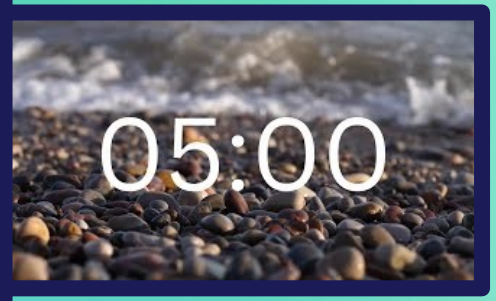
The **binary search** algorithm finds a target element in a **sorted list** by **dividing the list in half** in each iteration.



Target: 40

Break Time!

Take 5 mins to step away from the computer. Feel free to turn off your camera and return promptly after 5 mins.



60 minutes

Collaborative Problem Solving

Breakout Rooms

60 minutes

1. 🏃 Navigate to your breakout room.
2. 👋 Introduce yourselves!
3. 👦🏽💻 Work on the problems for your selected unit.

10 minutes

Wrap Up

One Good Thing

Let’s reflect on today’s session. Maybe you got a question answered, overcame a specific coding challenge, or learned how to do something completely new!

In the chat, share at least one good thing you’ve learned or experienced during the session today.

Let’s celebrate together!

Session Survey Time!

Final Reminders

- [ ] Finish the problem sets for additional practice
- [ ] Take the weekly HackerRank assessment by Sunday @ midnight
- [ ] Visit a **Study Hall Session** to deepen your understanding, work through challenges, and connect with course material alongside your peers!